

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Data Interpretation

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO4: *Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics. (BTL 5)*

Assessment Tool Weightage: 40%

Read the given data carefully and compare the techniques or results based on multiple factors such as accuracy, robustness, and computational constraints. Evaluate and justify your decision with appropriate reasoning considering the application context.

1. Real-Time System Design Trade-off

A surveillance system must operate under constraints:

- Viola-Jones: fast but less accurate (78%)
- YOLO: moderate speed, good accuracy (92%)
- Deep Learning: high accuracy (95%) but slow

Compare the techniques and evaluate which should be selected under strict latency constraints while maintaining acceptable accuracy. Justify your decision considering deployment conditions.

Python code:

```
methods = {'Viola-Jones': (78, 'fast'), 'YOLO': (92, 'moderate'), 'Deep Learning': (95, 'slow')}
chosen = [name for name, (acc, speed) in methods.items() if speed != 'slow' and acc >= 90]
print('Selected:', chosen[0])
print('Reason: 92% accuracy with moderate speed satisfies the latency constraint.')
```

Output:

Selected: YOLO

Reason: 92% accuracy with moderate speed satisfies the latency constraint.

2. Robust Detection under Environmental Variations

Detection accuracy across environments:

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Compare the robustness of each method and evaluate which technique provides the best overall performance across varying conditions. Justify trade-offs.

Python code:

```
data = {'Viola-Jones': [80, 60, 55], 'YOLO': [90, 75, 70], 'Deep Learning': [95, 85, 80]}
averages = {name: sum(values)/len(values) for name, values in data.items()}
for name, avg in averages.items(): print(f'{name}: {avg:.2f}% average accuracy')
print('Best overall robustness:', max(averages, key=averages.get))
```

Output:

```
Viola-Jones: 65.00% average accuracy
YOLO: 78.33% average accuracy
Deep Learning: 86.67% average accuracy
Best overall robustness: Deep Learning
```

3. Optical Flow Reliability vs Noise Sensitivity

Two optical flow methods show:

- Method A: stable vectors but misses small motions
- Method B: detects fine motion but noisy vectors

Compare both methods and evaluate which is more suitable for motion tracking in dynamic scenes. Justify based on application requirements.

Python code:

```
methods = {'Method A': {'stable': True, 'small_motion': False}, 'Method B': {'stable': False, 'small_motion': True}}
for name, result in methods.items():
    score = int(result['stable']) + int(result['small_motion'])
    print(f'{name}: suitability score = {score}/2')
print('Choice for dynamic scenes: Method B, with temporal smoothing to reduce noise.')
```

Output:

```
Method A: suitability score = 1/2
Method B: suitability score = 1/2
Choice for dynamic scenes: Method B, with temporal smoothing to reduce noise.
```

4. Background Modeling Strategy (GMM)

A GMM-based system shows:

- High false positives in dynamic background
- Missed detections in static background

Compare GMM performance across scenarios and evaluate whether GMM alone is sufficient.

Propose and justify improvements.

Python code:

```
problems = {'Dynamic background': 'high false positives', 'Static background': 'missed detections'}
improvements = ['shadow suppression', 'adaptive mixture updates', 'morphological filtering', 'background-subtraction plus a detector']
for condition, issue in problems.items(): print(condition + ': ' + issue)
print('GMM alone sufficient: No')
print('Improvements:', ', '.join(improvements))
```

Output:

```
Dynamic background: high false positives
Static background: missed detections
GMM alone sufficient: No
Improvements: shadow suppression, adaptive mixture updates, morphological filtering, background-subtraction plus a detector
```

5. Motion Estimation vs Computational Cost

Two methods:

- Method A: low error (5%), high computation
- Method B: moderate error (10%), low computation

Compare the methods and evaluate their suitability for real-time vs offline applications. Justify your decision.

Python code:

```
methods = {'Method A': {'error': 5, 'computation': 'high'}, 'Method B': {'error': 10, 'computation': 'low'}}
print('Real-time choice: Method B - lower computation, acceptable moderate error.')
print('Offline choice: Method A - lower error is more important than computation time.')
print('Error difference:', methods['Method B']['error'] - methods['Method A']['error'], 'percentage points')
```

Output:

```
Real-time choice: Method B - lower computation, acceptable moderate error.
Offline choice: Method A - lower error is more important than computation time.
Error difference: 5 percentage points
```

6. Structure from Motion Consistency

Two datasets produce:

- Dataset A: accurate depth but requires high-quality images
- Dataset B: inconsistent depth but works on low-quality images

Compare the datasets and evaluate which setup is more practical for real-world deployment.

Justify your reasoning.

Python code:

```
datasets = {'Dataset A': {'depth': 'accurate', 'quality': 'high'}, 'Dataset B': {'depth': 'inconsistent', 'quality': 'low'}}
for name, info in datasets.items(): print(f'{name}: depth={info["depth"]}, image requirement={info["quality"]}')
print('Deployment choice: Dataset B for broad real-world coverage; calibrate and filter its depth estimates.')
```

Output:

```
Dataset A: depth=accurate, image requirement=high
```

Dataset B: depth=inconsistent, image requirement=low

Deployment choice: Dataset B for broad real-world coverage; calibrate and filter its depth estimates.

7. Stereo Vision vs Motion Estimation

Depth estimation results:

- Stereo Vision: accurate but requires dual cameras
- Motion Estimation: less accurate but works with single camera

Compare both approaches and evaluate which is preferable under hardware constraints. Justify your choice.

Python code:

```
approaches = {'Stereo Vision': {'accuracy': 'high', 'cameras': 2}, 'Motion Estimation': {'accuracy': 'moderate', 'cameras': 1}}
for name, info in approaches.items(): print(f'{name}: accuracy={info["accuracy"]}, cameras={info["cameras"]}')
print('Hardware-constrained choice: Motion Estimation because it needs one camera.')
```

Output:

Stereo Vision: accuracy=high, cameras=2

Motion Estimation: accuracy=moderate, cameras=1

Hardware-constrained choice: Motion Estimation because it needs one camera.

8. Model Generalization vs Overfitting

Performance:

Model Training Testing

A	98%	70%
---	-----	-----

B	90%	88%
---	-----	-----

Compare the models and evaluate which is more reliable for deployment. Justify based on generalization and risk factors.

Python code:

```
models = {'A': (98, 70), 'B': (90, 88)}
for name, (train, test) in models.items(): print(f'Model {name}: train={train}%, test={test}%, gap={train-test} percentage points')
print('Deployment choice: Model B - smaller generalization gap and lower overfitting risk.')
```

Output:

Model A: train=98%, test=70%, gap=28 percentage points

Model B: train=90%, test=88%, gap=2 percentage points

Deployment choice: Model B - smaller generalization gap and lower overfitting risk.

9. YOLO Performance across Scales

Detection results:

- Large objects: 95%
- Medium objects: 85%
- Small objects: 60%

Compare performance across scales and evaluate whether YOLO alone is sufficient for multi-scale detection tasks. Justify improvements.

Python code:

```
accuracy = {'Large': 95, 'Medium': 85, 'Small': 60}
for scale, value in accuracy.items(): print(f'{scale} objects: {value}%')
print('Large-to-small drop:', accuracy['Large'] - accuracy['Small'], 'percentage points')
print('Improvement: multi-scale feature pyramids, small-object augmentation, and higher-resolution inputs.')
```

Output:

```
Large objects: 95%
Medium objects: 85%
Small objects: 60%
Large-to-small drop: 35 percentage points
Improvement: multi-scale feature pyramids, small-object augmentation, and higher-resolution inputs.
```

10. Motion Model Selection for Animation

Two models:

- Model A: smooth but physically inaccurate
- Model B: realistic but computationally intensive

Compare the models and evaluate which should be used for animation systems under different constraints (real-time vs cinematic). Justify your decision.

Python code:

```
models = {'A': {'smooth': True, 'realistic': False, 'cost': 'low'}, 'B': {'smooth': False, 'realistic': True, 'cost': 'high'}}
print('Real-time animation: Model A - smooth and computationally affordable.')
print('Cinematic animation: Model B - realism is worth the higher cost.')
```

Output:

```
Real-time animation: Model A - smooth and computationally affordable.
Cinematic animation: Model B - realism is worth the higher cost.
```

11. Detection Model Comparison under Resource Constraints

Performance:

Model	Accuracy	Latency
A	85%	40 ms
B	92%	90 ms
C	90%	60 ms

Compare the models and evaluate which is optimal for real-time systems with moderate accuracy requirements. Justify your decision.

Python code:

```
models = {'A': {'accuracy': 85, 'latency': 40}, 'B': {'accuracy': 92, 'latency': 90}, 'C': {'accuracy': 90, 'latency': 60}}
feasible = {k:v for k,v in models.items() if v['latency'] <= 60}
print('Feasible models (latency <= 60 ms):', list(feasible))
print('Selected model:', max(feasible, key=lambda k: feasible[k]['accuracy']))
print('Reason: 90% accuracy at 60 ms is a balanced real-time choice.')
```

Output:

```
Feasible models (latency <= 60 ms): ['A', 'C']
```

Selected model: C

Reason: 90% accuracy at 60 ms is a balanced real-time choice.

12. Robustness vs Speed Trade-off

Performance:

Method	Accuracy	Robustness	Speed
A	88%	High	Slow
B	85%	Moderate	Fast
C	90%	Low	Moderate

Compare the methods and evaluate the best choice for a balanced system. Justify trade-offs.

Python code:

```
methods = {'A': {'accuracy': 88, 'robustness': 'High', 'speed': 'Slow'}, 'B': {'accuracy': 85, 'robustness': 'Moderate', 'speed': 'Fast'}, 'C': {'accuracy': 90, 'robustness': 'Low', 'speed': 'Moderate'}}
rank = {'High': 3, 'Moderate': 2, 'Low': 1}
score = {k: v['accuracy'] + 3*rank[v['robustness']] + (2 if v['speed']=='Fast' else 1 if v['speed']=='Moderate' else 0) for k,v in methods.items()}
print('Balanced scores:', score)
print('Best balanced choice:', max(score, key=score.get))
```

Output:

Balanced scores: {'A': 97, 'B': 93, 'C': 94}

Best balanced choice: A

13. Optical Flow Stability Analysis

Results:

Method	Accuracy	Noise Sensitivity
A	High	High
B	Moderate	Low

Compare both methods and evaluate suitability for real-time tracking in noisy environments.

Python code:

```
methods = {'A': {'accuracy': 'High', 'noise_sensitivity': 'High'}, 'B': {'accuracy': 'Moderate', 'noise_sensitivity': 'Low'}}
for name, value in methods.items(): print(f'{name}: accuracy={value["accuracy"]}, noise_sensitivity={value["noise_sensitivity"]}')
print('Noisy real-time choice: Method B, because low noise sensitivity is more important than maximum clean-scene accuracy.')
```

Output:

A: accuracy=High, noise sensitivity=High

B: accuracy=Moderate, noise sensitivity=Low

Noisy real-time choice: Method B, because low noise sensitivity is more important than maximum clean-scene accuracy.

14. Motion Estimation Accuracy vs Efficiency

Results:

Method	Error	Computation
A	4%	High

B	9%	Low
C	6%	Moderate

Compare methods and evaluate the best approach for embedded systems.

Python code:

```
methods = {'A': {'error': 4, 'computation': 'High'}, 'B': {'error': 9, 'computation': 'Low'}, 'C': {'error': 6, 'computation': 'Moderate'}}
print('Lowest error:', min(methods, key=lambda k: methods[k]['error']))
print('Embedded-systems choice: Method C - moderate computation with lower error than Method B.')
print('Errors:', {k:v['error'] for k,v in methods.items()})
```

Output:

```
Lowest error: A
Embedded-systems choice: Method C - moderate computation with lower error than Method B.
Errors: {'A': 4, 'B': 9, 'C': 6}
```

15. GMM Performance in Different Conditions

Results:

Scenario	Accuracy
Static	92%
Dynamic	68%

Compare performance and evaluate whether GMM is reliable for all environments. Suggest improvements.

Python code:

```
accuracy = {'Static': 92, 'Dynamic': 68}
for condition, value in accuracy.items(): print(f'{condition}: {value}%')
print('Performance drop in dynamic scenes:', accuracy['Static'] - accuracy['Dynamic'], 'percentage points')
print('GMM reliable for all environments: No; combine it with adaptive modeling and a learned detector.')
```

Output:

```
Static: 92%
Dynamic: 68%
Performance drop in dynamic scenes: 24 percentage points
GMM reliable for all environments: No; combine it with adaptive modeling and a learned detector.
```

16. Object Detection under Scale Variation

Results:

Object Size	Accuracy
Large	94%
Medium	80%
Small	55%

Compare detection performance and evaluate system limitations. Justify improvements.

Python code:

```
accuracy = {'Large': 94, 'Medium': 80, 'Small': 55}
for size, value in accuracy.items(): print(f'{size}: {value}%')
```

```
print('Large-to-small drop:', accuracy['Large'] - accuracy['Small'], 'percentage points')
print('Improvements: feature pyramids, tiling, small-object augmentation, and higher-resolution inference.')
```

Output:

Large: 94%

Medium: 80%

Small: 55%

Large-to-small drop: 39 percentage points

Improvements: feature pyramids, tiling, small-object augmentation, and higher-resolution inference.

17. Stereo vs SfM Accuracy Comparison

Results:

Method	Accuracy	Hardware
Stereo	High	Dual camera
SfM	Moderate	Single camera

Compare and evaluate which is more practical under hardware limitations.

Python code:

```
methods = {'Stereo': {'accuracy': 'High', 'hardware': 'Dual camera'}, 'SfM': {'accuracy': 'Moderate', 'hardware': 'Single camera'}}
for name, value in methods.items(): print(f'{name}: {value["accuracy"]} accuracy, {value["hardware"]}')
print('Practical choice under hardware limitations: SfM, because it needs only one camera.')
```

Output:

Stereo: High accuracy, Dual camera

SfM: Moderate accuracy, Single camera

Practical choice under hardware limitations: SfM, because it needs only one camera.

18. Detection Model Generalization

Results:

Model	Training	Testing
A	97%	75%
B	90%	88%

Compare and evaluate which model is better for deployment.

Python code:

```
models = {'A': (97, 75), 'B': (90, 88)}
for name, (train, test) in models.items(): print(f'Model {name}: generalization gap = {train-test} points')
print('Deployment choice: Model B - test accuracy is higher and the gap is much smaller.')
```

Output:

Model A: generalization gap = 22 points

Model B: generalization gap = 2 points

Deployment choice: Model B - test accuracy is higher and the gap is much smaller.

19. Motion Tracking Accuracy Comparison

Results:

Method	Accuracy	Stability
--------	----------	-----------

A	High	Low
B	Moderate	High

Compare and evaluate which is suitable for long-term tracking.

Python code:

```
methods = {'A': {'accuracy': 'High', 'stability': 'Low'}, 'B': {'accuracy': 'Moderate', 'stability': 'High'}}
for name, value in methods.items(): print(f'{name}: accuracy={value["accuracy"]}, stability={value["stability"]}')
print('Long-term tracking choice: Method B, because high stability reduces drift over time.')
```

Output:

A: accuracy=High, stability=Low
 B: accuracy=Moderate, stability=High
 Long-term tracking choice: Method B, because high stability reduces drift over time.

20. Surveillance Model Selection

Results:

Model	Accuracy	Latency
A	82%	30 ms
B	90%	70 ms
C	95%	120 ms

Compare and evaluate under strict latency constraints.

Python code:

```
models = {'A': {'accuracy': 82, 'latency': 30}, 'B': {'accuracy': 90, 'latency': 70}, 'C': {'accuracy': 95, 'latency': 120}}
feasible = {k:v for k,v in models.items() if v['latency'] <= 70}
print('Feasible under strict latency:', list(feasible))
print('Selected model:', max(feasible, key=lambda k: feasible[k]['accuracy']))
print('Reason: Model B improves accuracy while staying below 70 ms.')
```

Output:

Feasible under strict latency: ['A', 'B']
 Selected model: B
 Reason: Model B improves accuracy while staying below 70 ms.

21. Noise Impact on Detection

Results:

Noise Level	Accuracy
Low	92%
Medium	80%
High	60%

Compare performance and evaluate system robustness.

Python code:

```
accuracy = {'Low': 92, 'Medium': 80, 'High': 60}
for level, value in accuracy.items(): print(f'{level} noise: {value}%')
print('Low-to-high noise decrease:', accuracy['Low'] - accuracy['High'], 'percentage points')
print('Robustness strategy: denoising, augmentation, and noise-aware training.')
```

Output:

Low noise: 92%

Medium noise: 80%

High noise: 60%

Low-to-high noise decrease: 32 percentage points

Robustness strategy: denoising, augmentation, and noise-aware training.

22. Depth Estimation Error Analysis

Results:

Method	Error
A	0.5 m
B	1.2 m
C	0.8 m

Compare and evaluate accuracy for real-world applications.

Python code:

```
error = {'A': 0.5, 'B': 1.2, 'C': 0.8}
for method, value in error.items(): print(f'Method {method}: {value:.1f} m error')
print('Most accurate method:', min(error, key=error.get))
print('Recommendation: Method A for real-world applications, subject to its computational cost.')
```

Output:

Method A: 0.5 m error

Method B: 1.2 m error

Method C: 0.8 m error

Most accurate method: A

Recommendation: Method A for real-world applications, subject to its computational cost.

23. Multi-Model Comparison

Results:

odel	Accuracy	Speed	Robustness
A	88%	Fast	Low
B	92%	Moderate	High
C	95%	Slow	High

Compare and evaluate the best overall model.

Python code:

```
models = {'A': {'accuracy': 88, 'speed': 'Fast', 'robustness': 'Low'}, 'B': {'accuracy': 92, 'speed': 'Moderate', 'robustness': 'High'}, 'C': {'accuracy': 95, 'speed': 'Slow', 'robustness': 'High'}}
for name, v in models.items(): print(f'{name}: {v["accuracy"]}%, {v["speed"]}, robustness={v["robustness"]}')
print('Best overall model: B - strong accuracy and robustness with moderate speed.')
```

Output:

A: 88%, Fast, robustness=Low

B: 92%, Moderate, robustness=High

C: 95%, Slow, robustness=High

Best overall model: B - strong accuracy and robustness with moderate speed.

24. Motion Detection under Crowd Density

Results:

Method	Sparse Crowd	Dense Crowd
Optical Flow	High	Low
Motion Models	Moderate	High

Compare and evaluate best approach.

Python code:

```
methods = {'Optical Flow': {'Sparse Crowd': 'High', 'Dense Crowd': 'Low'}, 'Motion Models': {'Sparse Crowd': 'Moderate', 'Dense Crowd': 'High'}}
for name, v in methods.items(): print(f'{name}: sparse={v["Sparse Crowd"]}, dense={v["Dense Crowd"]}')
print('Best general choice: Motion Models, because they handle dense crowds better.')
```

Output:

Optical Flow: sparse=High, dense=Low

Motion Models: sparse=Moderate, dense=High

Best general choice: Motion Models, because they handle dense crowds better.

25. Model Efficiency vs Accuracy

Results:

Model	Accuracy	Computation
A	90%	High
B	85%	Low
C	88%	Moderate

Compare and evaluate for edge devices.

Python code:

```
models = {'A': {'accuracy': 90, 'computation': 'High'}, 'B': {'accuracy': 85, 'computation': 'Low'}, 'C': {'accuracy': 88, 'computation': 'Moderate'}}
for name, v in models.items(): print(f'{name}: accuracy={v["accuracy"]}, computation={v["computation"]}')
print('Edge-device choice: Model C - a useful accuracy/efficiency compromise.')
```

Output:

A: accuracy=90%, computation=High

B: accuracy=85%, computation=Low

C: accuracy=88%, computation=Moderate

Edge-device choice: Model C - a useful accuracy/efficiency compromise.

26. Feature Detection under Illumination Changes

Results:

Method	Normal	Low Light
A	90%	60%
B	85%	75%

Compare and evaluate robustness.

Python code:

```
results = {'A': {'Normal': 90, 'Low Light': 60}, 'B': {'Normal': 85, 'Low Light': 75}}
for name, v in results.items(): print(f'{name}: normal={v["Normal"]}% , low-light={v["Low Light"]}%')
print('Robustness choice: Method B - smaller illumination drop of', results['B']['Normal']-results['B']['Low Light'], 'points.')
```

Output:

A: normal=90%, low-light=60%
 B: normal=85%, low-light=75%
 Robustness choice: Method B - smaller illumination drop of 10 points.

27. Video Processing Pipeline Comparison

Results:

Pipeline	Accuracy	Speed
A	88%	Fast
B	92%	Moderate
C	95%	Slow

Compare and evaluate suitability.

Python code:

```
pipelines = {'A': {'accuracy': 88, 'speed': 'Fast'}, 'B': {'accuracy': 92, 'speed': 'Moderate'}, 'C': {'accuracy': 95, 'speed': 'Slow'}}
for name, v in pipelines.items(): print(f'{name}: {v["accuracy"]}% , {v["speed"]}')
print('Suitability choice: Pipeline B - better accuracy than A without the slow speed of C.')
```

Output:

A: 88%, Fast
 B: 92%, Moderate
 C: 95%, Slow
 Suitability choice: Pipeline B - better accuracy than A without the slow speed of C.

28. Motion Model Evaluation

Results:

Model	Realism	Speed
A	High	Low
B	Moderate	High

Compare and evaluate based on application context.

Python code:

```
models = {'A': {'realism': 'High', 'speed': 'Low'}, 'B': {'realism': 'Moderate', 'speed': 'High'}}
for name, v in models.items(): print(f'{name}: realism={v["realism"]}, speed={v["speed"]}')
print('Real-time choice: Model B')
print('Offline/cinematic choice: Model A')
```

Output:

A: realism=High, speed=Low
 B: realism=Moderate, speed=High
 Real-time choice: Model B
 Offline/cinematic choice: Model A

29. Detection under Occlusion

Results:

Method	Accuracy
A	70%
B	85%
C	90%

Compare and evaluate robustness under occlusion.

Python code:

```
accuracy = {'A': 70, 'B': 85, 'C': 90}
for name, value in accuracy.items(): print(f'Method {name}: {value}% accuracy under occlusion')
print('Most robust method:', max(accuracy, key=accuracy.get))
print('Improvement: train with occlusion augmentation and use temporal tracking.')
```

Output:

```
Method A: 70% accuracy under occlusion
Method B: 85% accuracy under occlusion
Method C: 90% accuracy under occlusion
Most robust method: C
Improvement: train with occlusion augmentation and use temporal tracking.
```

30. System Performance under Constraints

Results:

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate
C	95%	120 ms	High

Compare and evaluate the best system considering all constraints.

Python code:

```
models = {'A': {'accuracy': 88, 'latency': 50, 'cost': 'Low'}, 'B': {'accuracy': 92, 'latency': 80, 'cost': 'Moderate'}, 'C':
{'accuracy': 95, 'latency': 120, 'cost': 'High'}}
feasible = {k:v for k,v in models.items() if v['latency'] <= 80 and v['cost'] != 'High'}
print('Feasible systems:', list(feasible))
print('Best constrained system:', max(feasible, key=lambda k: feasible[k]['accuracy']))
print('Reason: Model B gives 92% accuracy at 80 ms with moderate cost.')
```

Output:

```
Feasible systems: ['A', 'B']
Best constrained system: B
Reason: Model B gives 92% accuracy at 80 ms with moderate cost.
```

Code of Conduct:

*I S. Yuva Guru Mani Varma / 192425005 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *S. Yuva Guru Mani Varma*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult

		structure, visuals, and terminology			to understand
--	--	--	--	--	--------------------------